

Live session

Callbacks → async/await

One program, written three times, and then used properly

```
src/import.js
```

```
open(db, (e, db) => {  
  readFile(f, (e, raw) => {  
    insert(db, (e) => {  
      // ...  
    });  
  });  
});  
});  
  
const db = await open(f);  
const raw = await readFile(f);
```

The same program, three times

Seven steps. Every one of them asynchronous. Every one needs the one before it.

- 1 open the database, create the table
- 2 read a JSON file of readings
- 3 parse it
- 4 validate and insert every reading
- 5 ask for a summary per region
- 6 write the report
- 7 close the database

What changes

- Nothing about the problem.
- Nothing about the output — the report is identical, byte for byte.
- Only how the waiting is written.

Today's rule We are not learning three tools. We are looking at one machine through three syntaxes, and at why the third one won.

01

Callbacks

Correct code that is still unpleasant — and why that is not about taste

02 · Promises: two different jobs

Producing

- `new Promise((resolve, reject) => ...)`
- Only when wrapping an API that still speaks callbacks.
- You write this a handful of times in your career.
- Never wrap what the platform already wrapped: `node:fs/promises` exists.

Consuming

- `.then()` · `.catch()` · `.finally()`
- Using something that already returns a promise.
- This is what you write every day.
- A promise is a value: store it, pass it, attach handlers later.

The confusion: almost everybody who says "I don't get promises" is looking at producing code and thinking they need to write it.

03 · What a chain could not do

async/await compiles down to the same promises. So what actually changed?

Scope

In a chain, every `.then()` is its own function: a value from step 1 is not in scope in step 5. You carry it in a variable declared above, or you nest again. With `await`, it is simply a local variable.

Loops

With callbacks an asynchronous loop is a recursive function. With promises it is a `reduce`. With `await` it is a `for` loop — an actual one.

try / catch / finally

A rejection is an exception again, so the language constructs you already know work. The connection closes in a `finally`, once.

Not syntactic sugar the runtime is identical — what changed is what the language lets you write.

04

Concurrency

Until now every await was needed. Now some of them are not.

Sequential or concurrent?

One after another

- When each step needs the previous result.
- When the order of writes matters.
- Correct, and often the right answer.

All at once

- `Promise.all` — all must succeed.
- `Promise.allSettled` — report every outcome.
- Only when the work does not depend on itself.

The question to ask Does this step need the result of the one before it? If not, you are waiting for nothing — and if it does, there is nothing to optimise.

Three local files: 2 ms → 1 ms. Seven remote lookups: 1754 ms → 251 ms. Concurrency pays where the waiting is real.

KEY TAKEAWAYS

What to keep from this session

It was the same program all along.

Same seven steps, same report, same four failures. Only the waiting was written differently.

async/await won on scope and loops, not on length.

It is the same promises underneath — but variables and for loops work again.

Sequential or concurrent is a decision.

Ask whether the step needs the previous result. Everything else follows from that.

The code is in checkpoints/.

Open the four files side by side — that comparison is the whole session.